

Software Architecture Documentation

Project Name: Phoneword Generator

Due Date: February 12, 2025

1. System Overview

The **Phoneword Generator** is a Windows Forms application designed to convert numerical phone numbers into memorable word combinations, known as phonewords. The application allows users to:

- Input a mobile phone number.
 - Generate all possible letter combinations based on the telephone keypad mapping.
 - Identify and rank meaningful word combinations using a dictionary and custom word lists.
 - Perform reverse searches by converting words back into their numeric phone number equivalents.
 - Manage and export selected phonewords.
-

2. Architectural Style

The application follows a **Two-Layer Architecture**, comprising:

- 1. Presentation Layer (UI Layer):**
 - **Components:** Windows Forms interface (**MainForm**), UI controls, and event handlers.
 - **Responsibilities:**
 - Handle user input and interactions.
 - Display results and feedback.
 - Validate input data before processing.
 - Provide a responsive and user-friendly interface.
- 2. Business Layer:**
 - **Components:** Core classes such as **PhonewordService**, **PhonewordGenerator**, **PartitionGenerator**, **WordMatcher**, and **DigitLetterMapping**.

- **Responsibilities:**
 - Implement the core logic for phoneword generation and processing.
 - Handle data validation and exception management.
 - Perform algorithmic computations for generating and ranking phonewords.
 - Maintain separation of concerns, keeping business logic isolated from the UI.
-

3. Components and Their Interactions

3.1 Presentation Layer Components

- **MainForm:**
 - **Purpose:** Acts as the primary user interface.
 - **Key Responsibilities:**
 - Facilitates user interactions with the application.
 - Invokes methods from the [PhonewordService](#) in the Business Layer.
 - Handles input validation for phone numbers and words.
 - Displays generated phonewords and handles user actions such as adding to selected list, exporting, and clearing results.

3.2 Business Layer Components

- **PhonewordService:**
 - **Purpose:** Serves as a front for the Business Layer.
 - **Responsibilities:**
 - Coordinates the processes of phoneword generation, filtering, and ranking.
 - Provides methods for generating phonewords and performing reverse searches.
- **PhonewordGenerator:**
 - **Purpose:** Generates all possible phoneword combinations from a given phone number.
 - **Responsibilities:**
 - Generates partitions of the phone number.
 - Maps digits to letters.
 - Generates letter combinations based on partitions.
 - Implements limits to prevent excessive computation.

- **PartitionGenerator:**
 - **Purpose:** Generates all valid partitions of the phone number for different groupings of digits.
 - **Responsibilities:**
 - Recursively generates partitions of the phone number.
 - Handles special cases for digits '0' and '1'.
 - **WordMatcher:**
 - **Purpose:** Filters and ranks generated phonewords based on meaningfulness.
 - **Responsibilities:**
 - Compares generated phonewords against a dictionary and custom word list.
 - Ranks phonewords based on criteria such as whole word matches, number of partitions, and meaningful word length.
 - **DigitLetterMapping:**
 - **Purpose:** Provides a static mapping between digits and their corresponding letters on a telephone keypad.
 - **Responsibilities:**
 - Supplies the digit-to-letter mappings used during phoneword generation.
-

4. Data Flow and Control Flow

4.1 Generating Phonewords

1. **User Input:**
 - The user enters a phone number into the `txtPhoneNumber` textbox in the `MainForm`.
2. **Validation:**
 - `MainForm` validates the input to ensure it contains only digits.
3. **Initiate Generation:**
 - Upon clicking the "Generate" button, `MainForm` invokes the `GeneratePhonewords` method in `PhonewordService`.
4. **Partitioning:**
 - `PhonewordService` calls `GeneratePhonewords` in `PhonewordGenerator`.
 - `PhonewordGenerator` uses `PartitionGenerator` to generate all valid partitions of the phone number.
5. **Generating Combinations:**

- For each partition, `PhonewordGenerator` maps digits to letters using `DigitLetterMapping`.
- It generates all possible letter combinations based on the partitions.
- 6. Filtering and Ranking:**
 - The generated combinations are passed to `WordMatcher`.
 - `WordMatcher` filters combinations to identify those containing meaningful words from the dictionary or custom word list.
 - It ranks the phonewords based on defined criteria.
- 7. Displaying Results:**
 - The filtered and ranked phonewords are returned to `MainForm`.
 - `MainForm` displays the results in the `lstGenerated` list box.

4.2 Reverse Search

- 1. User Input:**
 - The user enters a word into the `txtWordInput` textbox in the `MainForm`.
 - 2. Validation:**
 - `MainForm` validates the word to ensure it contains only letters.
 - 3. Initiate Reverse Search:**
 - Upon clicking the "Reverse Search" button, `MainForm` invokes the `ReverseSearch` method in `PhonewordService`.
 - 4. Mapping Letters to Digits:**
 - `PhonewordService` uses `DigitLetterMapping` to map each letter of the word to its corresponding digit.
 - 5. Displaying Result:**
 - The numeric representation is returned to `MainForm`.
 - `MainForm` displays the result in the `lstGenerated` list box.
-

5. Detailed Class Descriptions

5.1 MainForm (Presentation Layer)

- **Attributes:**
 - `_dictionaryWords`: List of words loaded from the dictionary.
 - `_allGeneratedPhonewords`: List of all generated phonewords.
 - `_cancellationTokenSource`: Manages cancellation of asynchronous tasks.
- **Key Methods:**
 - `LoadDictionary()`: Loads words from a dictionary file.

- `btnGenerate_Click()`: Initiates phoneword generation.
- `btnReverseSearch_Click()`: Initiates reverse search.
- `GetCustomWords()`: Retrieves custom words entered by the user.
- Input validation event handlers for textboxes.
- Methods for managing UI elements and user interactions.

5.2 PhonewordService (Business Layer)

- **Purpose:** Facilitates communication between the MainForm and core logic classes.
- **Methods:**
 - `GeneratePhonewords(string number, IEnumerable<string> dictionary, IEnumerable<string> customWords, CancellationToken token)`: Generates and filters phonewords.
 - `ReverseSearch(string word)`: Converts a word to its numeric representation.

5.3 PhonewordGenerator (Business Layer)

- **Attributes:**
 - `MaxCombinations`: Maximum number of combinations allowed.
 - `_currentCombinationCount`: Tracks the number of combinations generated.
 - `_generationLimitReached`: Indicates if the combination limit is reached.
- **Methods:**
 - `GeneratePhonewords(string number, CancellationToken token)`: Generates phonewords based on the input number.
 - `GetCombinations(List<string> partition, CancellationToken token)`: Generates combinations for a partition.
 - `GetLettersForPart(string part, CancellationToken token)`: Retrieves possible letters for a part of the partition.

5.4 PartitionGenerator (Business Layer)

- **Attributes:**
 - `_number`: The phone number being partitioned.
- **Methods:**
 - `GeneratePartitions(CancellationToken token)`: Initiates recursive partition generation.
 - `GeneratePartitionsRecursive(string remaining, List<string> currentPartition, List<List<string>> result, CancellationToken token)`: Generates partitions recursively.

5.5 WordMatcher (Business Layer)

- **Attributes:**
 - `_dictionary`: Set of dictionary words.
 - `_customWords`: Set of custom words provided by the user.
- **Methods:**
 - `FilterAndRankPhonewords(List<string> phonewords, CancellationToken token)`: Filters and ranks phonewords based on meaningfulness and predefined criteria.

5.6 DigitLetterMapping (Business Layer)

- **Attributes:**
 - `_digitToLetters`: Dictionary mapping digits to letters.
 - **Methods:**
 - `GetLetters(char digit)`: Retrieves letters corresponding to a digit.
 - `GetMapping()`: Retrieves the entire digit-to-letter mapping dictionary.
-

6. Algorithms Explanation

6.1 Partition Generation Algorithm

- **Objective:** Generate all valid partitions of the input phone number to explore different groupings of digits.
- **Approach:**
 - Uses recursion to generate partitions (`GeneratePartitionsRecursive`).
 - Handles digits '0' and '1' specially, as they map to themselves.
 - For other digits, creates partitions of size 2 up to the length of the remaining number.
- **Example:**
 - Input number: "234"
 - Possible partitions:
 - ["2", "3", "4"]
 - ["2", "34"]
 - ["23", "4"]
 - ["234"]

6.2 Phoneword Generation Algorithm

- **Objective:** Generate all possible letter combinations from the partitions.
- **Approach:**
 - Maps each digit to possible letters using `DigitLetterMapping`.
 - Generates combinations of letters for each partition.
 - Uses a Cartesian product to combine letters from different partitions.
 - Implements a limit (`MaxCombinations`) to prevent excessive computations.
- **Example:**
 - Partition: ["23", "4"]
 - "23" maps to ["AD", "AE", "AF", "BD", "BE", "BF", "CD", "CE", "CF"]
 - "4" maps to ["G", "H", "I"]
 - Combinations: ["AD-G", "AD-H", "AD-I", "AE-G", ..., "CF-I"]

6.3 Word Matching and Ranking Algorithm

- **Objective:** Filter out nonsensical combinations and rank meaningful phonewords.
 - **Approach:**
 - Compares generated phonewords against the dictionary and custom words.
 - Assigns scores based on criteria:
 - **Whole Word Match:** Phoneword is a complete meaningful word.
 - **Total Partitions:** Fewer partitions are preferred.
 - **Meaningful Length:** Longer meaningful words contribute to a higher score.
 - **Meaningful Partitions:** More meaningful partitions increase the score.
 - Sorts the phonewords based on the assigned scores.
-

7. User Interface Design

- **Controls and Layout:**
 - **Input Fields:**
 - `txtPhoneNumber`: Textbox for entering the phone number.
 - `txtWordInput`: Textbox for entering a word for reverse search.
 - `txtCustomWords`: Textbox for entering custom words.
 - **Buttons:**
 - `btnGenerate`: Initiates phoneword generation.
 - `btnReverseSearch`: Initiates reverse search.
 - `btnCancel`: Cancels the ongoing generation process.

- **btnAdd**: Adds selected generated phonewords to the selected list.
 - **btnRemove**: Removes selected phonewords from the selected list.
 - **btnExport**: Exports selected phonewords to a text file.
 - **btnClearGenerated**: Clears the generated phonewords list.
 - **btnClearSelected**: Clears the selected phonewords list.
 - **btnClose**: Closes the application.
 - **List Boxes**:
 - **lstGenerated**: Displays generated phonewords.
 - **lstSelected**: Displays user-selected phonewords.
 - **Other Controls**:
 - **progressBar**: Indicates progress during phoneword generation.
 - **statusLabel**: Displays status messages to the user.
 - **txtSearchGenerated**: Textbox for searching within generated phonewords.
 - **Design Considerations**:
 - **Aesthetics**: A clean and professional interface.
 - **Usability**: Intuitive layout and responsive controls.
 - **Feedback**: Provides status updates and informative messages.
-

8. Error Handling and Validation

- **Input Validation**:
 - Ensures phone numbers contain only digits.
 - Validates that words contain only letters.
 - Filters out invalid characters in custom words.
 - **Exception Handling**:
 - Handles exceptions during file operations, like loading the dictionary.
 - Manages cancellation requests using **CancellationToken**.
 - Catches and displays errors that occur during phoneword generation.
 - **User Feedback**:
 - Displays error messages using message boxes.
 - Updates status messages to inform the user of the current operation.
 - Notifies the user when generation limits are reached or if the results are incomplete.
-

9. Future Enhancements

- **Performance Optimization:**
 - Implement multi-threading to enhance performance.
 - Optimize algorithms for faster computation.
- **Additional Features:**
 - Allow users to import custom dictionaries.
 - Support internationalization for multiple languages and character sets.
 - Enhance UI with responsive design and additional themes.